A Cross-Reference Stream Carver for Recovering Modern PDF Documents from Raw Disk Images

Itroduction and Problem :

File carving is the technique to recover a file from the storage device using only the content of bytes without the help of filesystem . it is important is forensic because the filesystem is often the first thing to go a disk is wiped or a partition table is overwritten or a investigator is handle only the raw image without or corrupted filesystem . when the filesystem directory entry are missing , the data may still present on the disk and the carver tries to find it and put it back together .

PDF file formet is one of the formet that used mostly in real word so being able to recover the a PDF from a damaged or unallocated region is genuinely useful the problem is that the pdf is structure formet not a simple stream of byte with fixed header or footer . the earliest carver scan only for the start signature and end signature and copied everything in between . that work for a clean jpeg but a pdf is a graph of object's tied together by cross reference table and a header to footer copy usually produces a file that no reader will open .

Cohen's 2007 work moved the field forward by treating the PDF as a collection of objects rather than one contiguous block . instead of copying bytes between two markers, it located the individual objects on disk and used the cross-reference table to stitch them back into a working document. That approach works far better with fragmentation and with partial overwrites.

The gap this tool addresses is what happened to PDF after 2007. From version 1.5 onward, the format allows the cross-reference data to be stored as a compressed stream object rather than as the old traitional table, and it allows many small objects to be packed together inside object streams. These compressed cross-reference streams are almost always Flate-compressed and very often run through a PNG-style predictor before compression to make them smaller. A carver that only understands the classic ASCII cross-reference table simply cannot read the map for these files. It either misses objects entirely or rebuilds a document that is incomplete. Today the great majority of PDFs produced by common software use cross-reference streams, so this is not an simple case.

The aim, then, is that We take the object-based carving idea and teach it to read cross-reference streams detect them, undo the predictor, decompress them, parse the entries, and follow those entries into the object streams so that the objects packed

inside can be recovered too. The result is a carver that can rebuild modern PDFs from a raw image even when the filesystem and the trailer are gone.

This is traditional xref table (Cross reference table)

Xref Object_id(Not written in pdf)

0 164

0000000000 65535 f 0

0000188037 00000 n 1

0000187899 00000 n 2

0000186689 00000 n 3

0000221300 00000 n 4

0000221672 00000 n 5

0000222004 00000 n 6

0000222390 00000 n 7

0000222763 00000 n 8

0000223099 00000 n 9

0000224288 00000 n 10

trailer

Obj	**Line**	**Means**
0	0000000000 65535 f	free entry. gen 65535. always head of free-object list. standard first row
1	0000188037 00000 n	in-use. object 1 at byte offset 188037
2	0000187899 00000 n	obj 2 at 187899
3	0000186689 00000 n	obj 3 at 186689
4	0000221300 00000 n	obj 4 at 221300
5	0000221672 00000 n	obj 5 at 221672
6	0000222004 00000 n	obj 6 at 222004
7	0000222390 00000 n	obj 7 at 222390

Obj	Line	Means
8	0000222763 00000 n	obj 8 at 222763
9	0000223099 00000 n	obj 9 at 223099
10	0000224288 00000 n	obj 10 at 224288
11	0000224646 00000 n	obj 11 at 224646

188037 this is the file offset number of the pdf it is neither the physical offset nor the logical offset it is file offset means a pdf first store in disk the first byte of pdf means it's header %PDF-1.x : file byte 0 is the file offset and 'f' means free and 'n' means it is in use why we use %512 rule because whenever the pdf store it generally store in sector/cluster boundry .

Modern pdf has stream xref that look like this :

79 0 obj → This xref is itself indirect object 79

<<

/Root 1 0 R → Document catalog = object 1

/Index [0 80] → Covers objects 0–79 (start 0, count 80)

/Size 80 → Covers objects 0–79 (start 0, count 80)

/Type /XRef → Declares cross-reference stream

/W [1 3 1] → Field widths: type=1B, field2=3B, field3=1B same tradi xref entry

/Filter /FlateDecode → Stream body zlib-compressed

/Length 262 → 262 compressed bytes follow stream

>>

Working of the System :

The diagram below gives the overall flow of the carver . Each box is one stage; the output of a stage feeds the next , This is the carving pipeline .

Raw disk image (input)

↓

Scan the image for PDF markers (%PDF , obj , stream , trailer , startxref) store in .idx file

↓

Locate the cross-reference data and decide its type: classic table or xref stream

↓

If xref stream: decompress it and reverse the PNG predictor to recover the raw rows and make .map file for each pdf

↓

Parse the cross-reference entries (object offsets and object-stream positions)

↓

Resolve every object, including those packed inside object streams

↓

Check the object graph and confirm the document opens

↓

Write the recovered PDF and a recovery report (output)

Artifact	**Scope**	**Made at step**	**Holds**
.idx	whole disk image	Scan (step 2)	every marker hit across image: %PDF, obj, stream, startxref + byte offsets. one file, all candidates
.map	one PDF	Parse → Resolve (steps 5–6)	that PDF's rebuilt xref: obj number → location, type, objstm parent. one per recovered pdf

Methodology: How the Tool Works

This section define the step by step process of the tool one stage at a time and explains what each part does and why it is there. The order follows the pipeline in Figure .

Scanning the image :

The tool reads the raw image and scan through it looking for the byte signatures that mark PDF structure: the pdf header, the start of an indirect object, the start of a stream,

the trailer, and the startxref keyword that points to the cross-reference data. Rather than search for each signature in a separate pass over the disk, the scan looks for all of them together in a single pass, which keeps it fast even on large images. As it goes, it records where every marker was found and builds an index of candidate object and cross-reference locations. Nothing is decoded yet at this stage; the point is only to know where the interesting bytes are.

Finding and reading the cross-reference data :

With the index in hand, the tool follows the startxref pointers, or, when those are missing, the locations of cross-reference objects found during the scan. For each candidate it has to decide which kind of cross-reference it is dealing with. An older PDF stores a plain-text table that lists, line by line, the byte offset of each object. A modern PDF instead stores a stream object whose type is XRef, and the same information is packed into binary rows inside that stream. The tool reads the stream's description of itself, in particular the field that gives the byte width of each column and the field that says which object numbers the rows cover, so that it knows exactly how to slice the decoded data later .

Decompressing and undoing the predictor :

Cross-reference streams are compressed, almost always with Flate. On their own, Flate streams are easy to decompress. The complication is the predictor. To squeeze the stream smaller, the producer often transforms each row of the cross-reference data before compression using a PNG-style predictor, where every byte is stored as the difference from a neighbouring byte rather than as its actual value. There are several predictor variants, and the row may use the one above it, the one to its left, the average of the two, or the Paeth function of its neighbours.

This is the heart of the contribution. After decompressing the stream, the tool walks through it row by row and reverses whichever predictor was applied, reconstructing the original byte values. A classic carver that skips this step is left with garbage rows and cannot read the offsets at all. Once the predictor is undone, the rows become clean and the cross-reference entries can be read directly.

Resolving objects, including object streams :

Each decoded entry describes one object. Most entries are simply a byte offset that says where to find the object in the image. But a cross-reference stream also allows a second kind of entry, which says that the object is not stored on its own at all but is packed inside another object, the object stream. For these, the tool opens the parent object stream, decompresses it, and pulls out the requested object by its position in that container. Handling both kinds of entry is what lets the tool recover the many small

objects that modern PDFs tuck away inside object streams, which a table-only carver never sees.

Recovering when the trailer is gone :

In a healthy PDF the trailer and the startxref pointer tell you where the cross-reference data lives. On a carved fragment those are frequently missing or overwritten. When that happens, the tool does not give up. Because the earlier scan already located cross-reference stream objects directly, it can fall back to those, decode them as above, and rebuild the object map from scratch without needing the trailer to point the way. This is what makes the approach hold up on real damaged evidence rather than only on clean files.

Validating the result :

Recovering bytes is not the same as recovering a pdf, so the last stage checks the work. The tool follows the recovered object graph from the document root through to the page objects and confirms that the structure hangs together and that the file actually opens. If only part of the document can be rebuilt, that is recorded honestly as a partial recovery rather than reported as a success. Deciding success by whether the document validates, rather than by counting raw matches, also avoids the inflated recovery numbers that come from counting the same content more than once.

Output :

Finally the tool writes out the reconstructed PDF together with a short report describing what was found, how many objects were resolved, how many came from object streams, and which files validated. The report is meant to be the kind of record an examiner can refer to later when explaining how a particular document was recovered.

Result's :

This tool as well as well known carver's scalpel photorec foremost and cohen tool was tested in publicly available forensic dataset's DFRWS 2007 forensic challenge image and the PDF material in the NIST Computer Forensic Reference Data Sets (CFReDS) and code generated synthetic forensic dataset for each dataset we compare the number of pdf actually present how many of them carved how many of them render and not render and how many pdf is duplicate.

The table are in excel file .

And for the precision recall and false positive calculation parameter or formula for the table is

Precision = Renderable / Total carve

Recall = Renderable / Actual present

False positive = Header only + Invalid / Total carve